request. So make sure to disable CSRF protection by skipping the `verify_authenticity_token` action.

ch10_11/rails/app/controllers/notification_tokens_controller.rb

```
  class NotificationTokensController < ApplicationController
»   before_action :authenticate_user!
»
»   skip_before_action :verify_authenticity_token

    def create
      current_user.notification_tokens.find_or_create_by!(notification_token)
    end

    private

    def notification_token
      params.require(:notification_token).permit(:token, :platform)
    end
  end
```

This is another example of how you're keeping your logic on the server. Traditional mobile apps would need their own authentication token or even a JWT to authorize network requests. Instead, you're reusing the existing web cookie across web and native. That simplifies your logic immensely—you only have one authentication strategy to manage. If the cookie ever expires or becomes invalidated, *both* web and native requests will correctly fail.

Wrap up this file by responding with a 201 status code, "Created," to let the apps know the record was created successfully.

ch10_12/rails/app/controllers/notification_tokens_controller.rb

```
  class NotificationTokensController < ApplicationController
    before_action :authenticate_user!

    skip_before_action :verify_authenticity_token

    def create
      current_user.notification_tokens.find_or_create_by!(notification_token)
```